

# Migration Automation

Series: AUTOM | Notebook: 8 of 8 | Created: January 2026

## Table of Contents

- 1. [Introduction](#)
- 2. [Migration Scenarios](#)
- 3. [Monaco Migration](#)
- 4. [Terraform Export](#)
- 5. [SaaS Upgrade Assistant](#)
- 6. [Validation and Verification](#)
- 7. [Summary](#)

## Prerequisites

Before starting this notebook, ensure you have:

| Requirement          | Description                        |
|----------------------|------------------------------------|
| Source tenant access | Admin access to source environment |
| Target tenant access | Admin access to target environment |
| API Tokens           | Tokens on both source and target   |
| Monaco or Terraform  | Migration tool installed           |

# Learning Objectives

---

By the end of this notebook, you will:

- Understand common migration scenarios
  - Know how to export and import configurations
  - Be able to handle non-portable configurations
  - Validate migration completeness
- 

## 1. Introduction

---

Configuration migration is the process of transferring Dynatrace settings from one environment to another. This is common in tenant consolidation, Managed-to-SaaS migration, and disaster recovery scenarios.

### Common Migration Scenarios

| Scenario                    | Description                               |
|-----------------------------|-------------------------------------------|
| <b>Managed to SaaS</b>      | Moving from self-hosted to Dynatrace SaaS |
| <b>Tenant Consolidation</b> | Merging multiple tenants into one         |
| <b>Environment Cloning</b>  | Creating dev/staging from production      |
| <b>Disaster Recovery</b>    | Restoring config to a new tenant          |
| <b>Config Backup</b>        | Periodic export for safekeeping           |

### The 90/10 Rule

**Important:** 90% of configurations migrate automatically, but the remaining 10% takes 90% of the effort.

Plan extra time for:

- Credentials and secrets

- Custom integrations
- Entity ID references
- Network-specific settings

---

## 2. Migration Scenarios

---

### What Can Be Migrated

| Configuration Type  | Portable | Notes                           |
|---------------------|----------|---------------------------------|
| Management Zones    | Yes      | Rules migrate, entity IDs don't |
| Auto-tagging Rules  | Yes      | Fully portable                  |
| Alerting Profiles   | Yes      | May need zone ID updates        |
| Dashboards          | Partial  | Entity IDs need updating        |
| SLOs                | Yes      | Metric expressions portable     |
| Synthetic Monitors  | Partial  | Location IDs may differ         |
| Request Attributes  | Yes      | Fully portable                  |
| Calculated Services | Yes      | Fully portable                  |

### What Cannot Be Migrated

| Configuration Type | Why Not                 | Action Required     |
|--------------------|-------------------------|---------------------|
| Credentials Vault  | Security isolation      | Re-enter manually   |
| API Tokens         | Tenant-specific         | Generate new tokens |
| Historic Data      | Not transferable        | Accept baseline gap |
| Cloud Integrations | Credential-dependent    | Reconfigure         |
| SSL Certificates   | Tenant-specific         | Re-upload           |
| Private Locations  | Infrastructure-specific | Redeploy            |

---

## Migration Tool Comparison

| Tool                   | Best For             | Pros                     | Cons                      |
|------------------------|----------------------|--------------------------|---------------------------|
| Monaco                 | Config-as-code teams | YAML-based, Git-friendly | Manual entity ID handling |
| Terraform Export       | IaC environments     | Full HCL output          | Large state files         |
| SaaS Upgrade Assistant | M2S migration        | Guided, UI-based         | Limited to M2S            |
| Settings API           | Custom scripts       | Full control             | Most manual effort        |

## 3. Monaco Migration

### Step 1: Download from Source

```
# Set source environment
export DT_SOURCE_URL="https://source-tenant.live.dynatrace.com"
export DT_SOURCE_TOKEN="&lt;your-source-api-token&gt;"

# Download all configurations
monaco download \
  --environment-url "$DT_SOURCE_URL" \
  --api-token "$DT_SOURCE_TOKEN" \
  --output-folder ./migration-export
```

### Step 2: Review and Clean

After download, review the exported configs:

```
# List exported configuration types
ls -la migration-export/

# Count configurations per type
find migration-export -name "config.yaml" | wc -l
```

## Configuration Cleanup

Remove or update:

| Item                      | Action                          |
|---------------------------|---------------------------------|
| Entity IDs                | Replace with selectors or names |
| Location IDs              | Map to target locations         |
| Credential references     | Update to target vault entries  |
| Environment-specific URLs | Update endpoints                |

## Step 3: Create Target Manifest

**manifest.yamll:**

```
manifestVersion: 1.0

projects:
  - name: migration
    path: migration-export

environmentGroups:
  - name: default
    environments:
      - name: target
        url:
          type: environment
          value: DT_TARGET_URL
        auth:
          token:
            type: environment
            value: DT_TARGET_TOKEN
```

## Step 4: Validate and Deploy

```
# Set target environment
export DT_TARGET_URL="https://target-tenant.live.dynatrace.com"
export DT_TARGET_TOKEN="&lt;your-target-api-token&gt;"

# Validate first
monaco validate manifest.yaml

# Dry run
monaco deploy manifest.yaml --environment target --dry-run

# Deploy
monaco deploy manifest.yaml --environment target
```

---

## Handling Entity ID References

Dashboards and alerting profiles often contain entity IDs. These need special handling:

### Option 1: Use Entity Selectors

Replace hardcoded IDs with selectors:

```
// Before (hardcoded ID)
{
  "entityId": "HOST-ABC123DEF456"
}

// After (selector)
{
  "entitySelector": "type(HOST),entityName(\"web-server-01\")"
}
```

### Option 2: Entity Mapping Script

```

import json
import re

def replace_entity_ids(config: dict, mapping: dict) -> dict:
    """Replace entity IDs using a source->target mapping."""
    config_str = json.dumps(config)

    for source_id, target_id in mapping.items():
        config_str = config_str.replace(source_id, target_id)

    return json.loads(config_str)

# Build mapping from entity names
entity_mapping = {
    "HOST-ABC123": "HOST-XYZ789",
    "SERVICE-DEF456": "SERVICE-UVW012"
}

```

---

## 4. Terraform Export

---

### Using the Export Utility

Dynatrace provides a Terraform export utility:

```

# Install the export utility
# Download from: https://github.com/dynatrace-oss/terraform-provider-dynatrace

# Export all configurations
terraform-provider-dynatrace-export \
    -e "$DT_SOURCE_URL" \
    -t "$DT_SOURCE_TOKEN" \
    -o ./terraform-export

```

## Export Output Structure

```
terraform-export/  
├─ main.tf  
├─ variables.tf  
├─ management_zones.tf  
├─ auto_tagging.tf  
├─ alerting.tf  
├─ dashboards/  
│   └─ *.tf  
└─ terraform.tfstate # Optional: import existing state
```

## Importing to Target

```
cd terraform-export  
  
# Update provider configuration for target  
cat &gt; terraform.tfvars &lt;&lt;EOF  
dynatrace_url    = "$DT_TARGET_URL"  
dynatrace_token  = "$DT_TARGET_TOKEN"  
EOF  
  
# Initialize and apply  
terraform init  
terraform plan  
terraform apply
```

---

## 5. SaaS Upgrade Assistant

---

For Managed-to-SaaS migrations, Dynatrace provides a guided tool.

### Accessing the Assistant

1. Log into your Dynatrace account
2. Navigate to **Apps → SaaS Upgrade Assistant**
3. Follow the guided workflow



## Assistant Features

| Feature             | Description                               |
|---------------------|-------------------------------------------|
| Discovery           | Automated inventory of source environment |
| Compatibility Check | Identify configs that need manual work    |
| Bulk Export         | Export all compatible settings            |
| Progress Tracking   | Visual dashboard of migration status      |
| Validation          | Post-migration validation checks          |

## When to Use

| Scenario        | Recommended Tool       |
|-----------------|------------------------|
| Managed to SaaS | SaaS Upgrade Assistant |
| SaaS to SaaS    | Monaco                 |
| Backup/Restore  | Monaco or Terraform    |
| GitOps workflow | Monaco                 |

# 6. Validation and Verification

## Pre-Migration Checklist

| Check                    | Description                      |
|--------------------------|----------------------------------|
| [ ] Source inventory     | Document all configurations      |
| [ ] Credential list      | Identify all secrets to re-enter |
| [ ] Entity dependencies  | Map entity ID references         |
| [ ] Network requirements | Verify target connectivity       |
| [ ] Token scopes         | Ensure sufficient permissions    |

## Post-Migration Validation

Run these DQL queries on the target tenant:

```
// Count management zones
fetch dt.settings
| filter schemaId == "builtin:management-zones"
| summarize count()
```

```
// Count auto-tagging rules
fetch dt.settings
| filter schemaId == "builtin:tags.auto-tagging"
| summarize count()
```

```
// Verify synthetic monitors
fetch dt.entity.synthetic_test
| summarize total = count()
```

## Validation Script

Compare source and target configuration counts:

```

import requests

def count_settings(url: str, token: str, schema_id: str) -> int:
    """Count settings objects of a given schema."""
    response = requests.get(
        f"{url}/api/v2/settings/objects",
        params={"schemaIds": schema_id},
        headers={"Authorization": f"Api-Token {token}"}
    )
    return response.json().get("totalCount", 0)

def validate_migration(source_url, source_token, target_url, target_token):
    """Compare configuration counts between tenants."""
    schemas = [
        "builtin:management-zones",
        "builtin:tags.auto-tagging",
        "builtin:alerting.profile",
        "builtin:slo"
    ]

    results = []
    for schema in schemas:
        source_count = count_settings(source_url, source_token, schema)
        target_count = count_settings(target_url, target_token, schema)

        results.append({
            "schema": schema,
            "source": source_count,
            "target": target_count,
            "match": source_count == target_count
        })

    return results

```

---

## Troubleshooting Common Issues

| Issue             | Cause               | Solution                                 |
|-------------------|---------------------|------------------------------------------|
| Missing configs   | Schema not exported | Export specific schema                   |
| Deploy errors     | Entity ID invalid   | Use selectors instead                    |
| Dashboard empty   | Data not migrated   | Expected (historic data doesn't migrate) |
| Alerts not firing | Credential issues   | Re-enter webhook credentials             |
| Synthetic failing | Location mismatch   | Update location IDs                      |

## 7. Summary

### Migration Best Practices

| Practice                   | Description                         |
|----------------------------|-------------------------------------|
| <b>Plan thoroughly</b>     | Document everything before starting |
| <b>Export first</b>        | Keep a backup of source config      |
| <b>Validate often</b>      | Check counts after each step        |
| <b>Test in dev</b>         | Validate in non-production first    |
| <b>Document exceptions</b> | Note what needed manual work        |

## Quick Reference: Migration Commands

| Task              | Monaco Command                                              |
|-------------------|-------------------------------------------------------------|
| Download all      | <code>monaco download --output-folder ./export</code>       |
| Download specific | <code>monaco download --api builtin:management-zones</code> |
| Validate          | <code>monaco validate manifest.yaml</code>                  |
| Dry run           | <code>monaco deploy manifest.yaml --dry-run</code>          |
| Deploy            | <code>monaco deploy manifest.yaml</code>                    |

## Series Complete

Congratulations! You've completed the AUTOM series. Here's what you learned:

| Notebook | Key Takeaway                            |
|----------|-----------------------------------------|
| AUTOM-01 | Choose the right tool for your use case |
| AUTOM-02 | Settings API is the foundation          |
| AUTOM-03 | Monaco for GitOps-style management      |
| AUTOM-04 | Terraform for state management          |
| AUTOM-05 | Workflows for event-driven automation   |
| AUTOM-06 | SDKs for custom applications            |
| AUTOM-07 | CI/CD for automated deployments         |
| AUTOM-08 | Migration patterns and validation       |

## Additional Resources

- [Monaco Documentation](#)
  - [Terraform Provider](#)
  - [Dynatrace API Reference](#)
  - [M2S Migration Series](#) - For Managed-to-SaaS specific guidance
-

**Key Takeaway:** Successful migration requires planning, the right tools, and thorough validation. Use Monaco or Terraform for bulk export/import, but always plan for manual work on credentials and entity references.

---

*This concludes the AUTOM series. For Managed-to-SaaS migrations, see the M2S series.*

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*